

Advanced method. The commitment $c = g_{\text{iden}}^{\text{id}} \cdot g_{\text{val}}^v \cdot g_{\text{ran}}^r$ and the public parameters pp are those of the Basic method, unchanged (Algorithm 0), so every Basic result continues to hold. As in the specification, each protocol is given in non-interactive form; the interactive form replaces the hash step with a challenge sent by the verifier, and the theorems carry over unchanged.

Protocol 3 Π.MIDEq (Multi-Commitment Identity-Equality Proof)

Parameter: Security parameter λ ; group parameters as in Protocol 1 (generators from HASHToGROUP, Algorithm 0) [P3]; two distinct domain separators, $\text{ds}_\gamma = \text{IBC}/\text{v1}/\text{MIDEq-}\gamma$ for the weights and $\text{ds}_{\text{MIDEq}} = \text{IBC}/\text{v1}/\text{MIDEq}$ for the challenge [P2].

Input: Commitments $c_i = g_{\text{iden}}^{\text{id}_i} \cdot g_{\text{val}}^{v_i} \cdot g_{\text{ran}}^{r_i}$ for $i = 1, \dots, n$ with $n \geq 2$. The prover holds every opening (id_i, v_i, r_i) , all with the same identity; the verifier holds $c_1, \dots, c_n$ and the context string ctx. [P2]

Output: Completeness with probability 1; zero-knowledge in the random-oracle model. The conclusion $\text{id}_1 = \dots = \text{id}_n$ follows only under DLR: against an adversary making Q random-oracle queries, commitments with valid openings where $\text{id}_j \neq \text{id}_1$ for some j are accepted with probability at most $(Q + 1)/p$ plus a negligible term (Theorem 3). [P1]

Prover	Verifier
1: for $i = 2, \dots, n$ do 2: Compute $\gamma_i = H(\text{ds}_\gamma, \text{pp}, c_1, \dots, c_n, \text{ctx}, i)$ 3: Compute $\Delta v = \sum_{i=2}^n \gamma_i (v_1 - v_i)$ $\Delta r = \sum_{i=2}^n \gamma_i (r_1 - r_i)$ 4: Randomly sample $\alpha_{\text{val}}, \alpha_{\text{ran}} \in \mathbb{Z}_p$ 5: Compute $t = g_{\text{val}}^{\alpha_{\text{val}}} \cdot g_{\text{ran}}^{\alpha_{\text{ran}}}$ 6: Compute $\beta = H(\text{ds}_{\text{MIDEq}}, \text{pp}, c_1, \dots, c_n, \text{ctx}, t)$ 7: Compute $z_{\text{val}} = \beta \Delta v + \alpha_{\text{val}}$ $z_{\text{ran}} = \beta \Delta r + \alpha_{\text{ran}}$ 8: Set $\pi = (t, z_{\text{val}}, z_{\text{ran}})$	9: for $i = 2, \dots, n$ do 10: Compute $\gamma_i = H(\text{ds}_\gamma, \text{pp}, c_1, \dots, c_n, \text{ctx}, i)$ 11: Compute $D = \prod_{i=2}^n (c_1/c_i)^{\gamma_i}$ 12: Compute $\beta = H(\text{ds}_{\text{MIDEq}}, \text{pp}, c_1, \dots, c_n, \text{ctx}, t)$ 13: if $g_{\text{val}}^{z_{\text{val}}} \cdot g_{\text{ran}}^{z_{\text{ran}}} = t \cdot D^\beta$ then 14: Return 1 (accept) 15: else 16: Return 0 (reject)

Every γ_i hashes all n commitments at once, so no weight is known before every c_i is fixed; a weight known in advance would let a cheater choose c_i with $\sum_i \gamma_i (\text{id}_1 - \text{id}_i) = 0$ while the identities differ. The proof is one group element and two scalars — 96 bytes for every n , against $96(n - 1)$ bytes for $n - 1$ separate Π.IDEq proofs. Steps 11 and 13 fuse into one multi-scalar multiplication over the $n + 3$ bases $g_{\text{val}}, g_{\text{ran}}, t, c_1, \dots, c_n$, in which c_1 carries the exponent $\beta \sum_i \gamma_i$ and every other c_i carries $-\beta \gamma_i$.

Protocol 4 Π.VEq (Hidden Value-Equality Proof)

Parameter: Security parameter λ ; group parameters as in Protocol 1 (generators from HASHToGROUP, Algorithm 0) [P3]; domain separator $\text{ds}_{\text{VEq}} = \text{IBC}/\text{v1}/\text{VEq}$ [P2].

Input: Commitments $c_b = g_{\text{iden}}^{\text{id}_b} \cdot g_{\text{val}}^{v_b} \cdot g_{\text{ran}}^{r_b}$ for $b = 0, 1$. The prover holds the openings (id_b, v_b, r_b) with $v_0 = v_1$; the identities may differ. The verifier holds c_0, c_1 and the context string ctx, and never learns the value. [P2]

Output: Completeness with probability 1; special soundness for the relation $c_0/c_1 = g_{\text{iden}}^{\Delta \text{id}} \cdot g_{\text{ran}}^{\Delta r}$, hence knowledge error $1/p$, and at most $(Q + 1)/p$ against Q random-oracle queries; zero-knowledge in the random-oracle model. The conclusion $v_0 = v_1$ follows only under DLR (Theorem 4). [P1]

Prover	Verifier
1: Randomly sample $\alpha_{\text{iden}}, \alpha_{\text{ran}} \in \mathbb{Z}_p$ 2: Compute $t = g_{\text{iden}}^{\alpha_{\text{iden}}} \cdot g_{\text{ran}}^{\alpha_{\text{ran}}}$ 3: Compute $\beta = H(\text{ds}_{\text{VEq}}, \text{pp}, c_0, c_1, \text{ctx}, t)$ 4: Compute $z_{\text{iden}} = \beta(\text{id}_0 - \text{id}_1) + \alpha_{\text{iden}}$ $z_{\text{ran}} = \beta(r_0 - r_1) + \alpha_{\text{ran}}$ 5: Set $\pi = (t, z_{\text{iden}}, z_{\text{ran}})$	6: Compute $\beta = H(\text{ds}_{\text{VEq}}, \text{pp}, c_0, c_1, \text{ctx}, t)$ 7: if $g_{\text{iden}}^{z_{\text{iden}}} \cdot g_{\text{ran}}^{z_{\text{ran}}} = t \cdot (c_0/c_1)^\beta$ then 8: Return 1 (accept) 9: else 10: Return 0 (reject)

This is Π.IDEq with the roles of g_{iden} and g_{val} exchanged: when $v_0 = v_1$ the g_{val} term of c_0/c_1 cancels. The witness $\text{id}_0 - \text{id}_1$ need not be zero, so the proof also links two different users — for instance, the amount sent equals the amount received. Nothing bounds the hidden value: both could be $p - 1$, which behaves like a negative amount, so either add a range proof or assume amounts are well-formed at issuance. Proof size 96 bytes.

Protocol 4.1 Π.FullEq (Re-Randomisation Proof)

Parameter: Security parameter λ ; group parameters as in Protocol 1 (generators from HASHToGROUP, Algorithm 0) [P3]; domain separator $\text{ds}_{\text{FullEq}} = \text{IBC}/\text{v1}/\text{FullEq}$ [P2].

Input: Commitments c_0, c_1 as in Protocol 4. The prover holds both openings, with $\text{id}_0 = \text{id}_1$ and $v_0 = v_1$, so $c_1 = c_0 \cdot g_{\text{ran}}^\rho$ with $\rho = r_1 - r_0$ is a re-randomisation of c_0 . The verifier holds c_0, c_1 and the context string ctx. [P2]

Output: As Theorem 4 with the single base g_{ran} : completeness with probability 1; special soundness for the relation $c_0/c_1 = g_{\text{ran}}^{\Delta r}$, hence knowledge error $1/p$, and at most $(Q + 1)/p$ against Q random-oracle queries; zero-knowledge in the random-oracle model. The conclusion $\text{id}_0 = \text{id}_1$ and $v_0 = v_1$ follows only under DLR. [P1]

Prover	Verifier
1: Randomly sample $\alpha_{\text{ran}} \in \mathbb{Z}_p$ 2: Compute $t = g_{\text{ran}}^{\alpha_{\text{ran}}}$ 3: Compute $\beta = H(\text{ds}_{\text{FullEq}}, \text{pp}, c_0, c_1, \text{ctx}, t)$ 4: Compute $z_{\text{ran}} = \beta(r_0 - r_1) + \alpha_{\text{ran}}$ 5: Set $\pi = (t, z_{\text{ran}})$	6: Compute $\beta = H(\text{ds}_{\text{FullEq}}, \text{pp}, c_0, c_1, \text{ctx}, t)$ 7: if $g_{\text{iden}}^{z_{\text{iden}}} \cdot g_{\text{ran}}^{z_{\text{ran}}} = t \cdot (c_0/c_1)^\beta$ then 8: Return 1 (accept) 9: else 10: Return 0 (reject)

The specification states Π.FullEq as a corollary of Protocol 4 and gives no separate listing; this is Protocol 4 with the single base g_{ran} . When identity and value both agree, the g_{iden} and g_{val} terms of c_0/c_1 both cancel, leaving $g_{\text{ran}}^{r_0 - r_1}$, so π proves that c_1 is a re-randomisation of c_0 without revealing ρ . One group element and one scalar — 64 bytes, the smallest proof in either method.

Protocol 5 Π.TxVer (Transaction-Verification Proof)

Parameter: Security parameter λ ; group parameters as in Protocol 1 (generators from HASHToGROUP, Algorithm 0) [P3]; domain separator $\text{ds}_{\text{TxVer}} = \text{IBC}/\text{v1}/\text{TxVer}$ [P2].

Input: A reference commitment c_{ref} issued at registration, which must be authenticated by the registering authority (the verifier checks this before step 6, outside the Sigma protocol), and a transaction commitment c_{tx} ; the prover holds their openings $(\text{id}_{\text{ref}}, v_{\text{ref}}, r_{\text{ref}})$ and $(\text{id}_{\text{tx}}, v_{\text{tx}}, r_{\text{tx}})$. The verifier holds $c_{\text{ref}}, c_{\text{tx}}$, the declared value v' and the context string ctx, which must include the transaction identifier. [P2]

Output: Completeness with probability 1; special soundness for the conjunction of the Π.IDEq and Π.VVer relations with knowledge error $1/p$ (two accepting transcripts sharing (t_1, t_2) yield both witnesses); zero-knowledge in the random-oracle model, where a proof for one tuple $(c_{\text{ref}}, c_{\text{tx}}, v', \text{ctx})$ is accepted for another with probability at most $(Q + 1)/p$. The conclusion $\text{id}_{\text{tx}} = \text{id}_{\text{ref}} \wedge v_{\text{tx}} = v'$ follows only under DLR (Theorem 5). [P1]

Prover	Verifier
1: Randomly sample $\alpha_{\text{val}}, \alpha_1, \alpha_{\text{iden}}, \alpha_2 \in \mathbb{Z}_p$ 2: Compute $t_1 = g_{\text{val}}^{\alpha_{\text{val}}} \cdot g_{\text{ran}}^{\alpha_1}$ $t_2 = g_{\text{iden}}^{\alpha_{\text{iden}}} \cdot g_{\text{ran}}^{\alpha_2}$ 3: Compute $\beta = H(\text{ds}_{\text{TxVer}}, \text{pp}, c_{\text{ref}}, c_{\text{tx}}, v', \text{ctx}, t_1, t_2)$ 4: Compute $z_{\text{val}} = \beta(v_{\text{ref}} - v_{\text{tx}}) + \alpha_{\text{val}}$ $z_1 = \beta(r_{\text{ref}} - r_{\text{tx}}) + \alpha_1$ $z_{\text{iden}} = \beta \cdot \text{id}_{\text{tx}} + \alpha_{\text{iden}}$ $z_2 = \beta \cdot r_{\text{tx}} + \alpha_2$ 5: Set $\pi = (t_1, t_2, z_{\text{val}}, z_1, z_{\text{iden}}, z_2)$	6: Compute $\beta = H(\text{ds}_{\text{TxVer}}, \text{pp}, c_{\text{ref}}, c_{\text{tx}}, v', \text{ctx}, t_1, t_2)$ 7: Compute $U_1 = c_{\text{ref}}/c_{\text{tx}}$ $U_2 = c_{\text{tx}} \cdot g_{\text{val}}^{-v'}$ 8: Set $b_1 = \left[g_{\text{val}}^{z_{\text{val}}} \cdot g_{\text{ran}}^{z_1} = t_1 \cdot U_1^\beta \right]$ 9: Set $b_2 = \left[g_{\text{iden}}^{z_{\text{iden}}} \cdot g_{\text{ran}}^{z_2} = t_2 \cdot U_2^\beta \right]$ 10: if $b_1 \wedge b_2$ then 11: Return 1 (accept) 12: else 13: Return 0 (reject)

U_1 and U_2 are the two target elements: b_1 is the identity clause (Π.IDEq between c_{ref} and c_{tx}) and b_2 the value clause (Π.VVer on c_{tx}); $[\cdot]$ is 1 when the equation holds and 0 otherwise. The masks α_1 and α_2 are distinct because the g_{ran} witness differs between the clauses: $r_{\text{ref}} - r_{\text{tx}}$ in the first, r_{tx} in the second. One challenge over both first messages makes the clauses inseparable. The proof is two group elements and four scalars — 192 bytes, the same as a separate Π.IDEq and Π.VVer; the gain is one hash instead of two, atomicity, and, once steps 8 and 9 are merged with a random weight, a single multi-scalar multiplication over the seven bases $g_{\text{iden}}, g_{\text{val}}, g_{\text{ran}}, t_1, t_2, c_{\text{ref}}, c_{\text{tx}}$. The transaction commitment c_{tx} and the context string ctx are different objects.

Algorithm 6 BatchVer (Batch Verification of Π.IDEq Proofs)

Parameter: Group parameters as in Protocol 1 (generators from HASHToGROUP, Algorithm 0) [P3]; weight length $\ell = 128$.

Input: m statements and proofs $(c_{0,j}, c_{1,j}, \text{ctx}_j, \pi_j)$ for $j = 1, \dots, m$, each π_j a Π.IDEq proof (Protocol 1) bound to its own context string ctx_j . BatchVer is run by the verifier alone; there is no prover. [P2]

Output: Accepts whenever every π_j is individually valid. If at least one π_j is invalid, accepts with probability at most $2^{-\ell}$ over the choice of $\delta_1, \dots, \delta_m$ (Theorem 6), or $Q \cdot 2^{-\ell}$ when the weights are hash-derived. [P1]

Verifier
1: for $j = 1, \dots, m$ do 2: Parse $\pi_j = (t_j, z_{\text{val},j}, z_{\text{ran},j})$ 3: Compute $\beta_j = H(\text{ds}_{\text{IDEq}}, \text{pp}, c_{0,j}, c_{1,j}, \text{ctx}_j, t_j)$ 4: Compute $D_j = c_{0,j}/c_{1,j}$ 5: Randomly sample $\delta_1, \dots, \delta_m \in \{1, \dots, 2^\ell\}$ 6: Compute $Z_{\text{val}} = \sum_{j=1}^m \delta_j z_{\text{val},j} \pmod{p}$ $Z_{\text{ran}} = \sum_{j=1}^m \delta_j z_{\text{ran},j} \pmod{p}$ 7: if $g_{\text{val}}^{Z_{\text{val}}} \cdot g_{\text{ran}}^{Z_{\text{ran}}} = \prod_{j=1}^m t_j^{\delta_j} \cdot \prod_{j=1}^m D_j^{\delta_j \beta_j}$ then 8: Return 1 (accept) 9: else 10: Return 0 (reject)

Step 7 is one multi-scalar multiplication over the $2m + 2$ bases $g_{\text{val}}, g_{\text{ran}}, t_1, \dots, t_m, D_1, \dots, D_m$, in place of m separate checks of four exponentiations each. Step 5 draws the weights as secret verifier randomness after all proofs are received, which gives the bound $2^{-\ell}$. The specification also allows deriving them from a hash of all m statements and proofs once these are fixed: verification is then deterministic and reproducible, and the bound becomes $Q \cdot 2^{-\ell}$. The report must state which is used. A rejected batch does not say which proof failed; fall back to individual verification or bisect the batch. The same algorithm covers Π.VVer, Π.VEq and Π.MIDEq by substituting the bases and D_j , and Π.TxVer by giving its two equations independent weights.